

Capítulo 5

The answers to the chapter review questions can be found in the Appendix.

1. Which statements about the `final` modifier are correct? (Choose all that apply.)

- A. Instance and `static` variables can be marked `final`.
- B. A variable is effectively `final` only if it is marked `final`.
- C. An object that is marked `final` cannot be modified.
- D. Local variables cannot be declared with type `var` and the `final` modifier.
- E. A primitive that is marked `final` cannot be modified.

Las instancias y los `static` si se pueden declarar `final` por lo que la opción A es correcta, E es correcta porque una vez declarado un primitivo como `final` ya es inmutable

2. Which of the following can fill in the blank in this code to make it compile? (Choose all that apply.)

```
public class Ant {  
    _____ void method() {}  
}
```

- A. `default`
- B. `final`
- C. `private`
- D. `Public`
- E. `String`
- F. `zzz:`

Esta permitido declarar un método de acceso como `private` por lo que C es correcta, B es correcta porque especifica el método como `final` y Java lo permite

3. Which of the following methods compile? (Choose all that apply.)

- A. `final static void rain() {}`
- B. `public final int void snow() {}`
- C. `private void int hail() {}`
- D. `static final void sleet() {}`
- E. `void final ice() {}`
- F. `void public slush() {}`

Final esta correctamente colocado en A y D puede ir tanto al inicio como después de `static` por eso ambas opciones son correctas

4. Which of the following can fill in the blank and allow the code to compile? (Choose all that apply.)

```
final _____ song = 6;
```

- A. `int`
- B. `Integer`
- C. `long`
- D. `Long`
- E. `double`
- F. `Double`

Directamente `int` es correcta ya que encaja sin ningún problema, `Integer` toma el `int` y hace autoboxing por lo que es correcto, al ser un `int` dentro de un `long` Java hace el cast de manera automática sin problema, en `double` un entero primitivo se puede promover automáticamente a un tipo de punto flotante de mayor tamaño

5. Which of the following methods compile? (Choose all that apply.)

- A. `public void january() { return; }`
- B. `public int february() { return null; }`
- C. `public void march() {}`
- D. `public int april() { return 9; }`
- E. `public int may() { return 9.0; }`
- F. `public int june() { return; }`

El tipo de retorno `void` no necesita devolver ningún valor por lo que la respuesta A es correcta, `void` incluso funciona con el `return` ya que al llegar a `{ }` Java asume que el código termino por eso C es correcta, D indica `int` y regresa un `int` por lo cual es correcta.

6. Which of the following methods compile? (Choose all that apply.)

- A. `public void violin(int... nums) {}`
- B. `public void viola(String values, int... nums) {}`
- C. `public void cello(int... nums, String values) {}`
- D. `public void bass(String... values, int... nums) {}`
- E. `public void flute(String[] values, ...int nums) {}`
- F. `public void oboe(String[] values, int[] nums) {}`

A es correcta porque cumple con el principio de ser el único parámetro, en B los dos parámetros están permitidos y es correcto porque el `varargs int` esta en la ultima posición, en F están bien declarados y no hay ningún problema de sintaxis

7. Given the following method, which of the method calls return 2? (Choose all that apply.)

```
public int juggle(boolean b, boolean... b2) {  
    return b2.length;  
}
```

- A. `juggle();`
- B. `juggle(true);`
- C. `juggle(true, true);`
- D. `juggle(true, true, true);`
- E. `juggle(true, {true, true});`
- F. `juggle(true, new boolean[2]);`

En D se asigna un true para b y los otros 2 para b2 por lo que compila correctamente, en F nuevamente se agrega un true a b y el arreglo [2] asigna los otros dos a b2

8. Which of the following statements is correct?

- A. Package access is more lenient than protected access.
- B. A public class that has private fields and package methods is not visible to classes outside the package.
- C. You can use access modifiers so only some of the classes in a package see a particular package class.
- D. You can use access modifiers to allow access to all methods and not any instance variables.
- E. You can use access modifiers to restrict access to all classes that begin with the word Test.

Teniendo en cuenta las jerarquías de acceso en java:

public → protected → package-private (default) → private

Describe el encapsulamiento puedes aplicar public a los métodos que desees y dejar private a tus variables de instancia protegiéndolas

9. Given the following class definitions, which lines in the `main()` method generate a compiler error? (Choose all that apply.)

// **Classroom.java**

```
package my.school;
public class Classroom {
    private int roomNumber;
    protected static String teacherName;
    static int globalKey = 54321;
    public static int floor = 3;
    Classroom(int r, String t) {
        roomNumber = r;
        teacherName = t; } }
```

// **School.java**

```
1: package my.city;
2: import my.school.*;
3: public class School {
4:     public static void main(String[] args) {
5:         System.out.println(Classroom.globalKey);
6:         Classroom room = new Classroom(101, "Mrs. Anderson");
7:         System.out.println(room.roomNumber);
8:         System.out.println(Classroom.floor);
9:         System.out.println(Classroom.teacherName); } }
```

A. None: the code compiles fine.

B. Line 5

C. Line 6

D. Line 7

E. Line 8

F. Line 9

En la línea 5 no se tiene acceso por lo que marca un error, en la línea 6 tampoco se tiene un modificador de acceso por lo tanto el alcance del código no llega, en la línea 7 `roomNumber` está declarado como privado, en la línea 9 como no se declara una herencia no se tiene acceso por lo tanto es un error de compilación

10. What is the output of executing the Chimp program?

// Rope.java

```
1: package rope;
2: public class Rope {
3:     public static int LENGTH = 5;
4:     static {
5:         LENGTH = 10;
6:     }
7:     public static void swing() {

8:         System.out.print("swing ");
9:     } }
```

// Chimp.java

```
1: import rope.*;
2: import static rope.Rope.*;
3: public class Chimp {
4:     public static void main(String[] args) {
5:         Rope.swing();
6:         new Rope().swing();
7:         System.out.println(LENGTH);
8:     } }
```

- A. swing swing 5
- B. swing swing 10**
- C. Compiler error on line 2 of Chimp
- D. Compiler error on line 5 of Chimp
- E. Compiler error on line 6 of Chimp
- F. Compiler error on line 7 of Chimp

Todo compila correctamente, imprimiendo swing y la variable LENGHT

11. Which statements are true of the following code? (Choose all that apply.)

```
1: public class Rope {  
2:     public static void swing() {  
3:         System.out.print("swing");  
4:     }  
5:     public void climb() {  
6:         System.out.println("climb");  
7:     }  
8:     public static void play() {  
9:         swing();  
10:        climb();  
11:    }  
12:    public static void main(String[] args) {  
13:        Rope rope = new Rope();  
14:        rope.play();  
15:        Rope rope2 = null;  
16:        System.out.print("-");  
17:        rope2.play();  
18:    } }
```

- A. The code compiles as is.
- B. There is exactly one compiler error in the code.
- C. There are exactly two compiler errors in the code.
- D. If the line(s) with compiler errors are removed, the output is swing-climb.
- E. If the line(s) with compiler errors are removed, the output is swing-swing.
- F. If the line(s) with compiler errors are removed, the code throws a NullPointerException.

El primer error es que los métodos static no pueden mandar llamar variables de referencia, si se elimina la línea 10 el código se ejecuta sin problema

12. How many variables in the following method are effectively final?

```
10: public void feed() {  
11:     int monkey = 0;  
12:     if(monkey > 0) {  
13:         var giraffe = monkey++;  
14:         String name;  
15:         name = "geoffrey";  
16:     }  
17:     String name = "milly";  
18:     var food = 10;  
19:     while(monkey <= 10) {  
20:         food = 0;  
21:     }  
22:     name = null;  
23: }
```

- A. 1
- B. 2**
- C. 3
- D. 4
- E. 5
- F. None of the above. The code does not compile.

Solo existen dos variables que no se modifican a lo largo del código por eso mismo la respuesta correcta es B

13. What is the output of the following code?

```
// RopeSwing.java
import rope.*;
import static rope.Rope.*;
public class RopeSwing {
    private static Rope rope1 = new Rope();
    private static Rope rope2 = new Rope();
    {
        System.out.println(rope1.length);
    }

    public static void main(String[] args) {
        rope1.length = 2;
        rope2.length = 8;
        System.out.println(rope1.length);
    }
}

// Rope.java
package rope;
public class Rope {
    public static int length = 0;
}
```

- A. 02
- B. 08
- C. 2
- D. 8**
- E. The code does not compile.
- F. An exception is thrown.

El bloque de instancia no hace nada y el truco estático hace que rope1.length y rope2.length modifiquen exactamente la misma variable. Por eso, el programa solo imprime 8

14. How many lines in the following code have compiler errors?

```
1: public class RopeSwing {  
2:     private static final String leftRope;  
3:     private static final String rightRope;  
4:     private static final String bench;  
5:     private static final String name = "name";  
6:     static {  
7:         leftRope = "left";  
8:         rightRope = "right";  
9:     }  
10:    static {  
11:        name = "name";  
12:        rightRope = "right";  
13:    }  
14:    public static void main(String[] args) {  
15:        bench = "bench";  
16:    }  
17: }
```

- A. 0
- B. 1
- C. 2
- D. 3
- E. 4**
- F. 5

Name se declara dos veces, bench no se asigna nunca por eso mismo la línea 4 no compila, la línea 15 porque no se asignan las variables, rightRope se declara dos veces

15. Which of the following can replace line 2 to make this code compile? (Choose all that apply.)

```
1: import java.util.*;
2: // INSERT CODE HERE
3: public class Imports {
4:     public void method(ArrayList<String> list) {
5:         sort(list);
6:     }
7: }
```

- A. `import static java.util.Collections;`
- B. `import static java.util.Collections.*;`
- C. `import static java.util.Collections.sort(ArrayList<String>);`
- D. `static import java.util.Collections;`
- E. `static import java.util.Collections.*;`
- F. `static import java.util.Collections.sort(ArrayList<String>);`

Sintaxis correcta e importa todo por lo que B es correcta

16. What is the result of the following statements?

```
1: public class Test {
2:     public void print(byte x) {
3:         System.out.print("byte-");
4:     }
5:     public void print(int x) {
6:         System.out.print("int-");
7:     }
8:     public void print(float x) {
9:         System.out.print("float-");
10:    }
11:    public void print(Object x) {
12:        System.out.print("Object-");
13:    }
14:    public static void main(String[] args) {
15:        Test t = new Test();
16:        short s = 123;
17:        t.print(s);
18:        t.print(true);

19:        t.print(6.789);
20:    }
21: }
```

- A. byte-float-Object-
- B. int-float-Object-
- C. byte-Object-float-
- D. int-Object-float-
- E. int-Object-Object-**
- F. byte-Object-Object-

En la línea 17 al ser un short se convierte en int, el argumento de la línea 19 es un double y pasa como un double por lo que llama el print de la 11, el argumento de la línea 18 es un boolean y pasa como un boolean por lo que llama al print de la línea 11

17. What is the result of the following program?

```
1: public class Squares {  
2:     public static long square(int x) {  
3:         var y = x * (long) x;  
4:         x = -1;  
5:         return y;  
6:     }  
7:     public static void main(String[] args) {  
8:         var value = 9;  
9:         var result = square(value);  
10:        System.out.println(value);  
11:    } }
```

- A. -1
- B. 9**
- C. 81
- D. Compiler error on line 9
- E. Compiler error on a different line

Value nunca se reasigna nuevamente por lo que su valor se queda en 9

18. Which of the following are output by the following code? (Choose all that apply.)

```
public class StringBuilders {  
    public static StringBuilder work(StringBuilder a,  
        StringBuilder b) {  
        a = new StringBuilder("a");  
        b.append("b");  
        return a;  
    }  
    public static void main(String[] args) {  
        var s1 = new StringBuilder("s1");  
        var s2 = new StringBuilder("s2");  
  
        var s3 = work(s1, s2);  
        System.out.println("s1 = " + s1);  
        System.out.println("s2 = " + s2);  
        System.out.println("s3 = " + s3);  
    }  
}
```

- A. s1 = a
- B. s1 = s1
- C. s2 = s2
- D. s2 = s2b
- E. s3 = a
- F. The code does not compile.

S1 nunca cambio su referencia en memoria por lo que la opción B es correcta, s2 fue modificado con el append por lo que D es correcta, s3 recibio el return de a por lo que E es correcta

19. Which of the following will compile when independently inserted in the following code?
(Choose all that apply.)

```
1: public class Order3 {  
2:     final String value1 = "red";  
3:     static String value2 = "blue";  
4:     String value3 = "yellow";  
5:     {  
6:         // CODE SNIPPET 1  
7:     }  
8:     static {  
9:         // CODE SNIPPET 2  
10:    } }
```

- A. Insert at line 6: `value1 = "green";`
- B. Insert at line 6: `value2 = "purple";`
- C. Insert at line 6: `value3 = "orange";`
- D. Insert at line 9: `value1 = "magenta";`
- E. Insert at line 9: `value2 = "cyan";`
- F. Insert at line 9: `value3 = "turquoise";`

En B un bloque de instancia puede acceder y modificar variables estáticas, como value 2 no es final permite hacer cambios

En C value3 es una variable de instancia común dentro de un bloque de instancia. Se puede modificar sin problemas.

En E value2 es una variable static. Un bloque static puede acceder y modificar perfectamente variables estáticas.

20. Which of the following are true about the following code? (Choose all that apply.)

```
public class Run {  
    static void execute() {  
        System.out.print("1-");  
    }  
    static void execute(int num) {  
        System.out.print("2-");  
    }  
  
    static void execute(Integer num) {  
        System.out.print("3-");  
    }  
    static void execute(Object num) {  
        System.out.print("4-");  
    }  
    static void execute(int... nums) {  
        System.out.print("5-");  
    }  
    public static void main(String[] args) {  
        Run.execute(100);  
        Run.execute(100L);  
    }  
}
```

- A. The code prints out 2-4-.
- B. The code prints out 3-4-.
- C. The code prints out 4-2-.
- D. The code prints out 4-4-.
- E. The code prints 3-4- if you remove the method `static void execute(int num)`.
- F. The code prints 4-4- if you remove the method `static void execute(int num)`.

Al imprimir 100 llama al valor int por lo que imprime 2- por eso A es correcta, 100L es un long como Java no puede hacer un autoboxing a un tipo mas pequeño se llama a método Object por lo que E es correcta

21. Which method signatures are valid overloads of the following method signature? (Choose all that apply.)

`public void moo(int m, int... n)`

- A. `public void moo(int a, int... b)`
- ☒ B. `public int moo(char ch)`
- C. `public void moooo(int... z)`
- ☒ D. `private void moo(int... x)`
- E. `public void moooo(int y)`
- F. `public void moo(int... c, int d)`
- G. `public void moo(int... i, int j...)`

Es correcta porque hacen sobrecargas validas, con nombres diferentes y con sintaxis correctas